DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

2026–27

ODD SEMESTER

INFORMATION ASSURANCE AND SECURITY

(24CS3105)

ALM – PROTOTYPE REVIEW

submitted by

P. Vyshnavy
2420090021

Cluster: 4 Section: 11

Course Instructor

Dr. Jagan Mohan Reddy Danda
Associate Professor



KONERU LAKSHMAIAH EDUCATION FOUNDATION, BOWRAMPET

Hyderabad–500043, Telangana, India

2026–27

Abstract

This report presents two educational projects developed as part of the Information Assurance and Security course: CryptoVault and SecureTunnel. CryptoVault is a web-based cryptography toolkit built using React.js and a Python Flask backend. It demonstrates RSA encryption and decryption, Diffie–Hellman key exchange, digital signature generation and verification, SHA-512 hashing, and X.509 certificate generation. SecureTunnel is an interactive network-security visualizer built using React.js and Flask. It presents the conceptual stages of a TLS handshake, SSH connection establishment, and the differences between IPsec Transport Mode and Tunnel Mode. The projects provide an interactive way to study cryptographic operations and secure-communication concepts. The current SecureTunnel implementation is an educational visualization rather than a live TLS or SSH connection. The report describes the objectives, architecture, implementation, testing approach, limitations, and future enhancements of both projects.

Contents

Abstract **i**

1 Introduction **1**
1.1 Objectives 1

2 Project 1: CryptoVault **1**
2.1 Project Overview 1
2.2 Objectives 1
2.3 Technologies Used 2
2.4 System Architecture 2
2.5 RSA Encryption and Decryption 2
2.6 Diffie–Hellman Key Exchange 3
2.7 Digital Signature 3
2.8 SHA-512 Hashing 3
2.9 X.509 Certificate 3
2.10 CryptoVault API and Features 4
2.11 Prototype Screenshot 4

3 Project 2: SecureTunnel **4**
3.1 Project Overview 4
3.2 Objectives 5
3.3 Technologies Used 5
3.4 System Architecture 5
3.5 TLS Handshake Visualization 6
3.6 SSH Connection Visualization 6
3.7 IPsec Transport Mode 7
3.8 IPsec Tunnel Mode 7
3.9 API Endpoints 7
3.10 Prototype Screenshot 8

4 Testing and Results **8**
4.1 Running the Applications 9

5 Comparison of the Two Projects **9**

6 Limitations and Future Enhancements **9**
6.1 CryptoVault 9
6.2 SecureTunnel 10

7 Conclusion **10**

8 Source Code Repository **10**

List of Tables

1 Technologies used in CryptoVault 2
2 Technologies used in SecureTunnel 5
3 Comparison of IPsec modes 7
4 SecureTunnel API endpoints 7
5 CryptoVault test cases 8
6 SecureTunnel test cases 8
7 CryptoVault and SecureTunnel comparison 9

List of Figures

1 CryptoVault system architecture 2
2 CryptoVault user interface 4
3 SecureTunnel system architecture 5
4 SecureTunnel user interface 8

1 Introduction

Information Assurance and Security focuses on protecting information and communication systems from unauthorized access, modification, and disclosure. Cryptography provides mechanisms for confidentiality, integrity, authentication, and secure key establishment. Secure communication protocols use these mechanisms to protect data exchanged over networks.

This report documents two projects. CryptoVault demonstrates selected cryptographic algorithms and operations through a web interface. SecureTunnel provides a visual explanation of TLS, SSH, and IPsec concepts.

1.1 Objectives

The main objectives are:

- To demonstrate encryption, decryption, hashing, and digital signatures.
- To understand public-key cryptography and key exchange.
- To provide a web interface for interacting with cryptographic operations.
- To visualize the conceptual stages of secure network protocols.
- To compare IPsec Transport Mode and Tunnel Mode.
- To test frontend-backend communication using REST API endpoints.

2 Project 1: CryptoVault

2.1 Project Overview

CryptoVault is a web-based cryptography toolkit. The frontend is developed using React.js, while the backend uses Python Flask to process requests and perform cryptographic operations. The application provides a user interface for experimenting with public-key cryptography, hashing, signatures, key exchange, and certificates.

2.2 Objectives

- To implement RSA encryption and decryption.
- To demonstrate Diffie–Hellman shared-secret establishment.
- To generate and verify digital signatures.

- To calculate SHA-512 message digests.
- To generate a self-signed X.509 certificate for demonstration.

2.3 Technologies Used

Table 1: Technologies used in CryptoVault

Technology	**Purpose**
React.js	Frontend interface and interactive controls
JavaScript, HTML, CSS	User interaction and page styling
Python	Backend implementation
Flask	REST API endpoints
Python Cryptography library	Cryptographic operations and certificate handling
Flask-CORS	Cross-origin requests from the frontend

2.4 System Architecture

The user interacts with the React.js frontend. The frontend sends requests to the Flask backend through HTTP REST API endpoints. The backend validates the request, performs the requested operation, and returns the result as JSON. The frontend displays the output to the user.

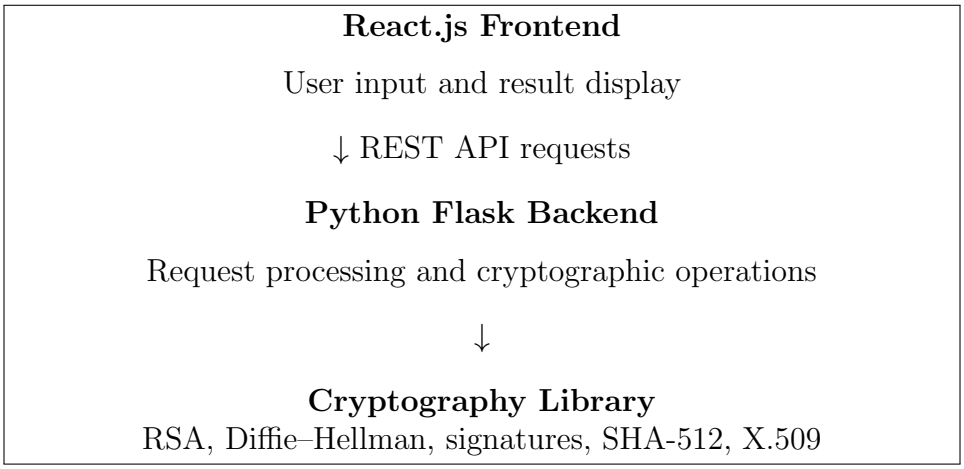


Figure 1: CryptoVault system architecture

2.5 RSA Encryption and Decryption

RSA is an asymmetric cryptographic algorithm that uses a public key and a private key. The public key is used for encryption and the corresponding private key is used for decryption. In this project, RSA operations are performed by the backend and the output is returned to the frontend.

For textbook RSA, the mathematical representation is:

$$C = M^e \bmod n \tag{1}$$

$$M = C^d \bmod n \tag{2}$$

where M is the message represented numerically, C is the ciphertext, (e, n) is the public key, and (d, n) is the private key. A practical implementation should use secure padding, such as RSA-OAEP for encryption, rather than textbook RSA directly.

2.6 Diffie–Hellman Key Exchange

Diffie–Hellman is a key-establishment method that enables two parties to derive a shared secret over a communication channel. Let p be a public prime and g a public generator. Alice chooses private value a and Bob chooses private value b.

$$A = g^a \bmod p \tag{3}$$

$$B = g^b \bmod p \tag{4}$$

Alice computes $S = B^a \bmod p$, while Bob computes $S = A^b \bmod p$. Both calculations produce the same shared secret when the parameters are valid. Diffie–Hellman by itself does not authenticate the communicating parties, so it must be combined with authentication in a real system.

2.7 Digital Signature

A digital signature is used to verify message integrity and the identity associated with the signing key. The sender signs a message using a private key, and the recipient verifies the signature using the corresponding public key. If the message or signature is altered, verification should fail. CryptoVault demonstrates signature generation and verification through its frontend and backend.

2.8 SHA-512 Hashing

SHA-512 is a cryptographic hash function that produces a 512-bit digest, usually displayed as 128 hexadecimal characters. It can be used to check whether data has changed. Hashing is a one-way operation and is not decryption or encryption.

2.9 X.509 Certificate

An X.509 certificate associates an identity with a public key and includes information such as the subject, issuer, validity period, and certificate signature. CryptoVault demon-

strates certificate generation. A self-signed certificate is useful for learning, but it is not automatically trusted by browsers or other systems.

2.10 CryptoVault API and Features

The implemented interface includes operations for RSA encryption and decryption, Diffie–Hellman key exchange, digital signature generation and verification, SHA-512 hashing, and X.509 certificate generation. The exact API endpoint names depend on the current backend implementation.

2.11 Prototype Screenshot

Upload screenshots of your running CryptoVault interface to the Overleaf project and replace the filename below with the exact uploaded filename.

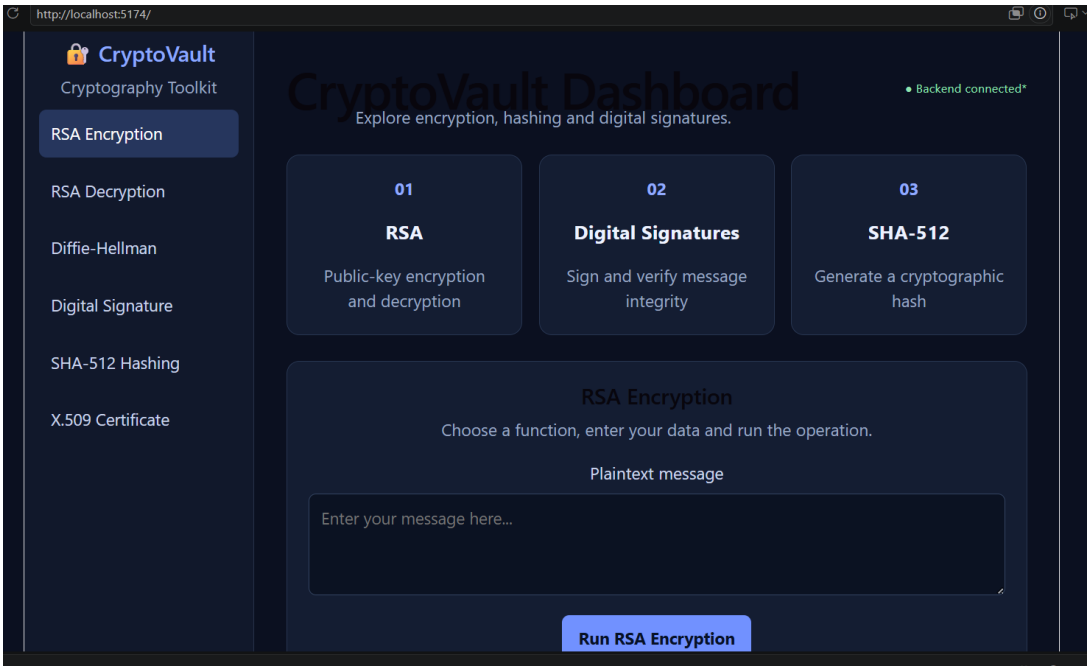


Figure 2: CryptoVault user interface

3 Project 2: SecureTunnel

3.1 Project Overview

SecureTunnel is an educational network-security visualizer developed using React.js and Python Flask. It allows users to explore the conceptual stages of TLS handshake establishment, SSH connection setup, and the two main IPsec modes. The frontend uses interactive navigation controls, while the backend returns protocol descriptions through JSON API responses.

3.2 Objectives

- To explain the main stages of a TLS handshake.
- To demonstrate the conceptual stages of an SSH connection.
- To compare IPsec Transport Mode and Tunnel Mode.
- To implement communication between React.js and Flask.
- To provide an accessible visual learning tool for network security.

3.3 Technologies Used

Table 2: Technologies used in SecureTunnel

Technology	**Purpose**
React.js	Interactive frontend and protocol navigation
JavaScript, HTML, CSS	Interface logic and styling
Python	Backend implementation
Flask	REST API endpoints for protocol information
Flask-CORS	Cross-origin frontend-backend communication
Vite	Frontend development server and build tooling

3.4 System Architecture

The React frontend provides navigation for TLS, SSH, and IPsec sections. It requests information from the Flask backend. The backend returns the protocol stages or mode descriptions as JSON. The frontend then displays the returned data in an interactive format.

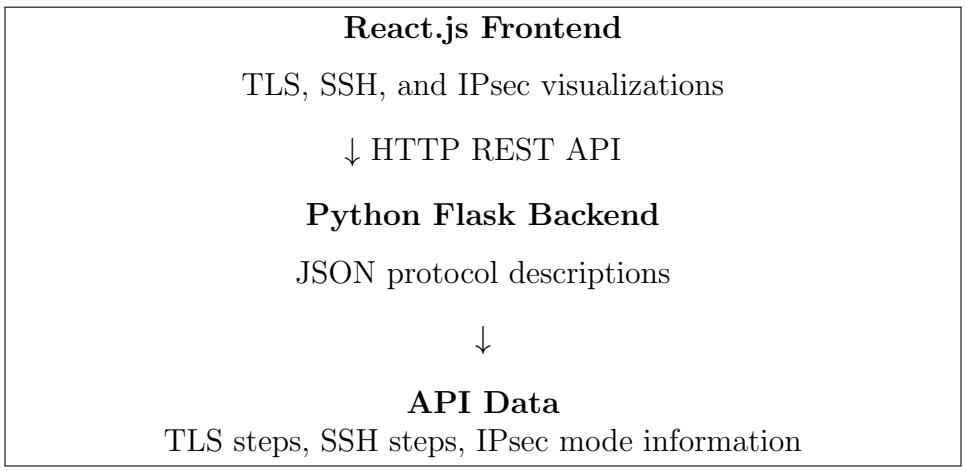


Figure 3: SecureTunnel system architecture

3.5 TLS Handshake Visualization

Transport Layer Security (TLS) is a protocol for protecting communication between clients and servers. The SecureTunnel interface presents a simplified sequence of handshake concepts:

1. **Client Hello:** The client proposes supported protocol versions and cryptographic parameters.
2. **Server Hello:** The server selects compatible parameters.
3. **Certificate:** The server can provide a certificate to support authentication.
4. **Key Establishment:** The parties establish shared session secrets using the negotiated key-establishment mechanism.
5. **Finished:** The handshake transcript is verified and the connection can proceed with protected application data.

The frontend provides Previous and Next Step controls to navigate the stages. This is a conceptual visualization; it does not execute a real TLS handshake. Actual TLS 1.3 message sequences can differ from this simplified teaching sequence.

3.6 SSH Connection Visualization

Secure Shell (SSH) is a protocol used for secure remote login and command execution. The SecureTunnel interface illustrates these conceptual stages:

1. **Protocol Version Exchange:** Client and server identify the SSH protocol version.
2. **Key Exchange:** The parties negotiate algorithms and establish shared session keys.
3. **Server Authentication:** The client checks the server's host key.
4. **User Authentication:** The user proves identity using a supported authentication method.
5. **Secure Session:** An encrypted session is used for commands and other supported SSH channels.

The stages are displayed for educational purposes. The application does not connect to an actual SSH server or perform real user authentication.

3.7 IPsec Transport Mode

In Transport Mode, IPsec protects the payload of an IP packet while retaining the original IP header. This mode is commonly used for communication between hosts. The protection provided depends on whether IPsec uses Authentication Header (AH) or Encapsulating Security Payload (ESP) and on the configured security association.

3.8 IPsec Tunnel Mode

In Tunnel Mode, the original IP packet is encapsulated inside a new IP packet. The outer header is used to route the packet between tunnel endpoints. This mode is commonly used in VPN gateway-to-gateway or host-to-gateway scenarios.

Table 3: Comparison of IPsec modes

Feature	**Transport Mode**	**Tunnel Mode**
Protected content	IP payload	Entire original IP packet
IP header	Original header remains	New outer header is added
Common use	Host-to-host communication	VPN tunnels and gateways

3.9 API Endpoints

The current frontend is configured to request the following endpoints from the Flask backend:

Table 4: SecureTunnel API endpoints

Endpoint	**Purpose**
`/api/tls`	Returns TLS handshake stages
`/api/ssh`	Returns SSH connection stages
`/api/ipsec`	Returns Transport Mode and Tunnel Mode descriptions

The frontend code also expects the backend to run on port 5001. Confirm that the Flask application uses this port and that the routes match before testing.

3.10 Prototype Screenshot

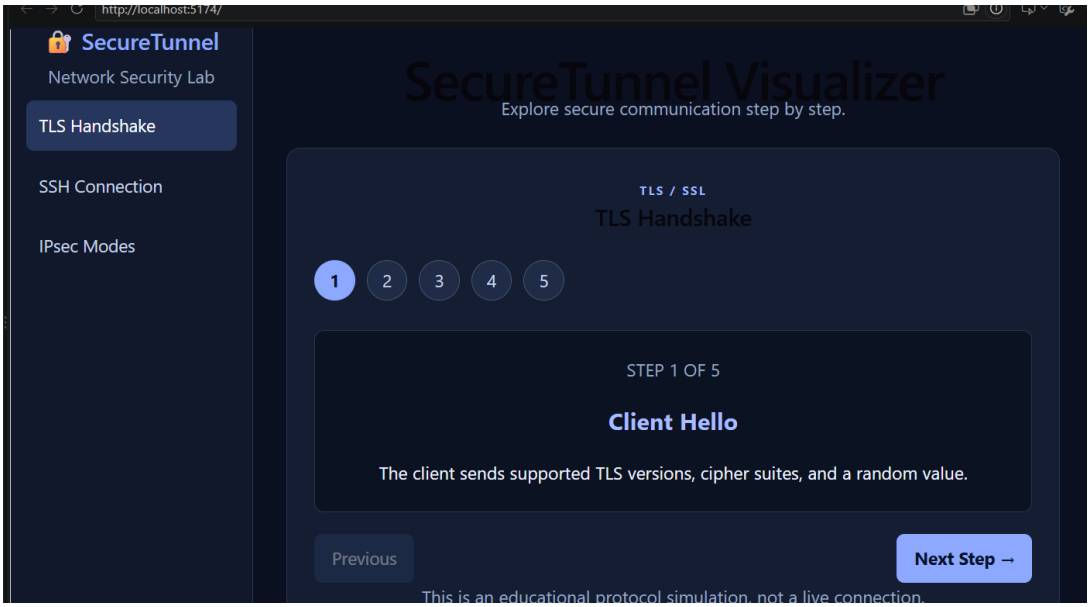


Figure 4: SecureTunnel user interface

4 Testing and Results

The applications should be tested with their respective backend and frontend servers running. The following tables provide test cases to record the observed results. Mark a test as passed only after performing it.

Table 5: CryptoVault test cases

Test case	**Expected result**	**Status**
RSA encryption	Ciphertext is generated from input	To be tested
RSA decryption	Original message is recovered	To be tested
Diffie–Hellman	Both sides derive the same shared secret	To be tested
Digital signature	Valid signature verifies successfully	To be tested
SHA-512	A 512-bit digest is displayed	To be tested
X.509 certificate	Certificate information is generated	To be tested

Table 6: SecureTunnel test cases

Test case	**Expected result**	**Status**
Backend root route	Running status or API response is shown, if implemented	To be tested
TLS endpoint	TLS stage data is returned as JSON	To be tested

Test case	Expected result	Status
SSH endpoint	SSH stage data is returned as JSON	To be tested
IPsec endpoint	Both IPsec mode descriptions are returned	To be tested
Frontend navigation	User can navigate between sections and steps	To be tested

4.1 Running the Applications

For CryptoVault, open separate terminals for the Flask backend and React frontend. Start the backend using the Python executable in its virtual environment and start the frontend using `npm.cmd run dev` from the frontend directory. The current project configuration uses port 5000 for the CryptoVault backend.

For SecureTunnel, start the Flask backend from the backend directory using its virtual-environment Python executable, and start the frontend using `npm.cmd run dev` from the frontend directory. The frontend code provided for SecureTunnel expects the backend at `http://127.0.0.1:5001`. Use the actual ports configured in the current source files if they differ.

5 Comparison of the Two Projects

Table 7: CryptoVault and SecureTunnel comparison

Aspect	CryptoVault	SecureTunnel
Main focus	Cryptographic operations	Network security concepts
Frontend	React.js	React.js
Backend	Python Flask	Python Flask
Main features	RSA, Diffie–Hellman, signatures, SHA-512, X.509	TLS visualization, SSH visualization, IPsec modes
Interaction	Submit input and inspect cryptographic output	Navigate protocol stages and compare modes
Implementation type	Cryptographic operation demonstrator	Educational protocol visualizer

6 Limitations and Future Enhancements

6.1 CryptoVault

The current project is an educational prototype. Key persistence, production key management, user authentication, access control, and deployment hardening may not be implemented. Future enhancements include secure key storage, authenticated access, input

validation, stronger operational error handling, and additional cryptographic demonstrations.

6.2 SecureTunnel

The current TLS and SSH sections visualize protocol stages rather than performing real TLS handshakes or SSH sessions. The IPsec section compares mode descriptions rather than configuring IPsec tunnels. Future enhancements could include real TLS connection inspection, integration with a controlled SSH test server, richer packet diagrams, and an interactive VPN demonstration in a safe test environment.

7 Conclusion

CryptoVault and SecureTunnel provide complementary demonstrations of Information Assurance and Security. CryptoVault allows users to explore cryptographic operations including RSA encryption and decryption, Diffie–Hellman key exchange, digital signatures, SHA-512 hashing, and X.509 certificate generation. SecureTunnel explains TLS handshake concepts, SSH connection stages, and IPsec operating modes through an interactive web interface. These projects support practical learning of cryptographic and network-security fundamentals. Further development can improve authentication, key management, protocol realism, testing, and deployment security.

8 Source Code Repository

CryptoVault repository :

https://github.com/vyshnavy2025/IAS_2420090021

SecureTunnel repository:

https://github.com/vyshnavy2025/IAS_2420090021